

Module - 3



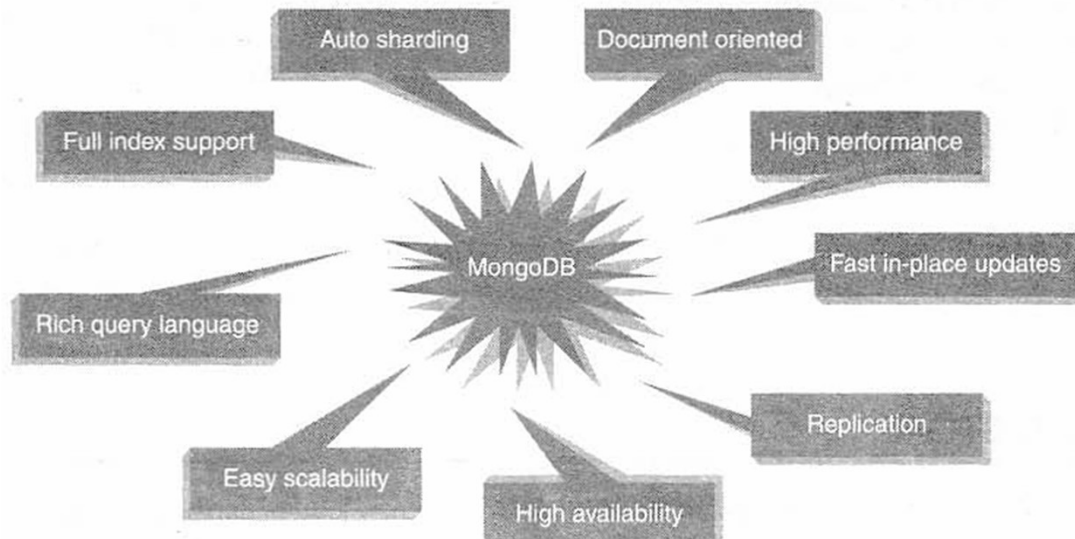
Introduction to MongoDB: What is MongoDB, Why MongoDB, Terms used in RDBMS and MongoDB, Data Types in MongoDB, MongoDB Query Language.

TB1: Ch 6: 6.1-6.5

What is MongoDB?

- MongoDB is
 1. **Cross-platform**: It works on different operating systems like **Windows, Linux, and macOS**. You can develop on one OS and run on another without issues.
 2. **Open source**: Anyone can download MongoDB and start using it without paying.
 3. **Non relational**: Unlike traditional databases (like MySQL), MongoDB doesn't use **tables and rows**. Instead, it stores data in **documents**, which are more flexible.
 4. **Distributed**: MongoDB can run on **multiple servers** to handle large amounts of data. This improves **speed, availability, and reliability**.
 5. **NoSQL**: "NoSQL" means **Not Only SQL**. It doesn't follow the strict structure of SQL databases. You can store **unstructured, semi-structured, or structured data** easily.
 6. **Document oriented data source**: Data is stored in **documents** (like JSON format). Each document can have different fields and structure, giving **flexibility** to store real-world data.

Why MongoDB?



Auto Sharding

- MongoDB **automatically breaks large data into smaller pieces (shards)** and **stores them** across **multiple servers**.
- Helps in **handling large-scale data** and **improves performance**.

Document Oriented

- **Stores data in documents** (like JSON), making it **easy to read and flexible** to **handle** different types of data.

High Performance

MongoDB is optimized for **speed and efficiency**, **even with large amounts of data**.

Fast In-Place Updates

You can quickly **update only the required part of a document** without replacing the whole thing.

Replication

- MongoDB **creates copies of data (replicas)** on **multiple servers**.
- This **keeps the data safe and available** even if one server fails.

High Availability

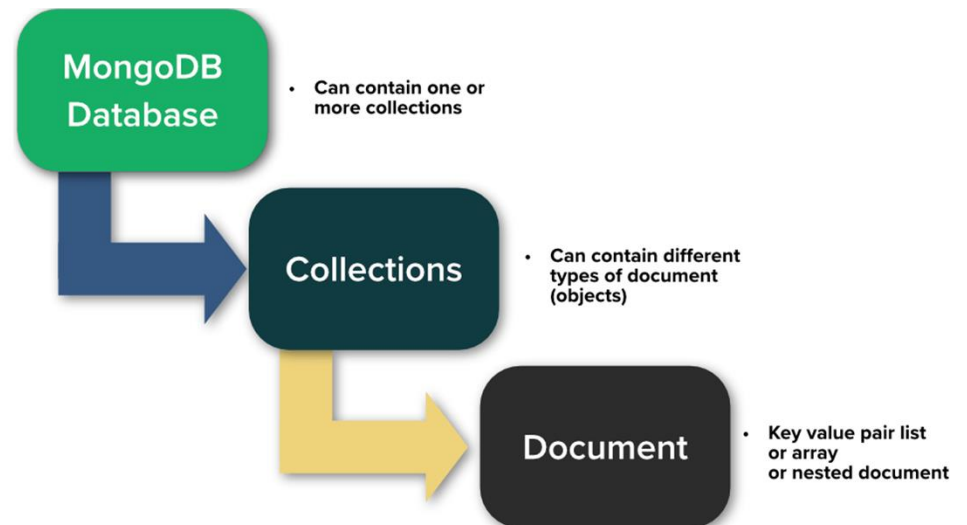
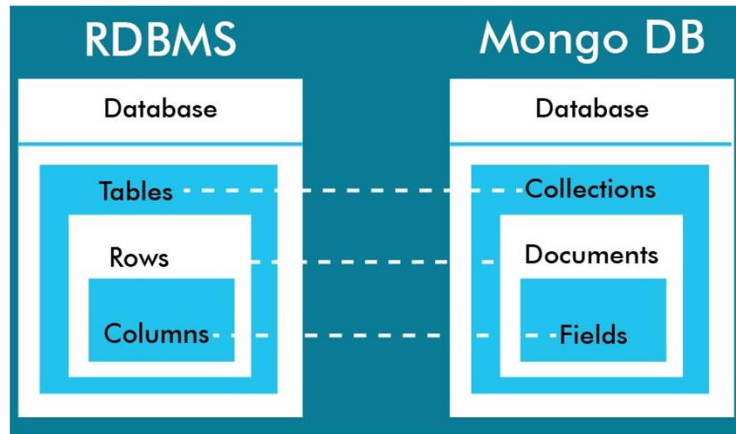
With features like **replication and automatic failover**, MongoDB ensures that **data is always accessible**.

Easy Scalability: You can **easily grow the database as your data grows**. **Add more servers without major changes** to your application.

Rich Query Language: MongoDB allows **powerful and flexible queries**, similar to SQL but suited for documents.

Full Index Support

Just like traditional databases, MongoDB **supports indexing to make data search faster**.

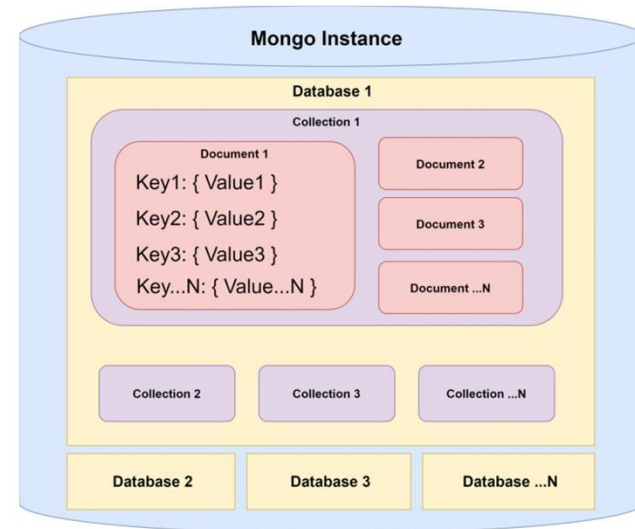


```

{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
  
```

Embedded sub-document

Embedded sub-document



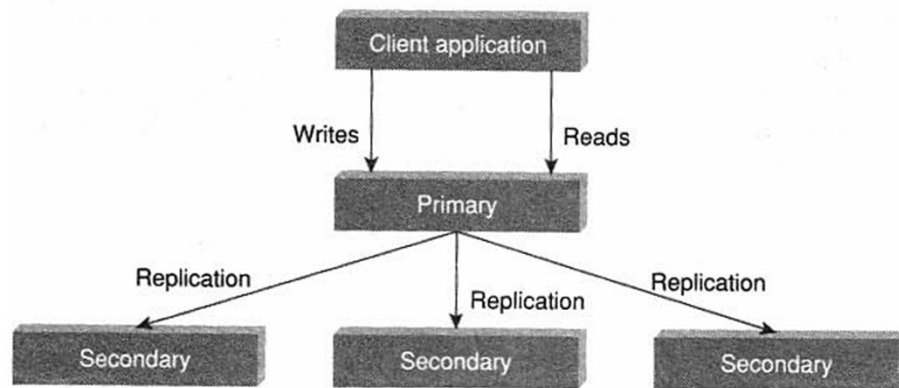


Figure 6.3 The process of **REPLICATION** in MongoDB.

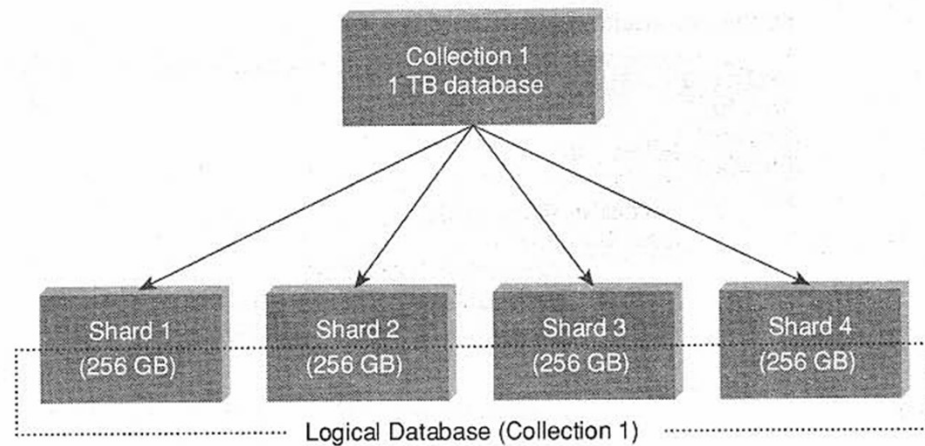


Figure 6.4 The process of **SHARDING** in MongoDB.

Terms used in RDBMS and MongoDB

RDBMS	MongoDB
Database	Database
Table	Collection
Record	Document
Columns	Fields / Key Value pairs
Index	Index
Joins	Embedded documents
Primary Key	Primary key (_id is a identifier)

	MySQL	Oracle	MongoDB
Database Server	MySql	Oracle	Mongod
Database Client	MySql	SQL Plus	mongo

MongoDB	RDBMS
It is a non-relational and document-oriented database.	It is a relational database.
It is suitable for hierarchical data storage.	It is not suitable for hierarchical data storage.
It has a dynamic schema.	It has a predefined schema.
It centers around the CAP theorem (Consistency, Availability, and Partition tolerance).	It centers around ACID properties (Atomicity, Consistency, Isolation, and Durability).
In terms of performance, it is much faster than RDBMS.	In terms of performance, it is slower than MongoDB.

Create a database

The syntax for creating database is as follows:

```
use DATABASE_Name
```

To create a database by the name “myDB” the syntax is

```
use myDB
```

```
> use myDB;
switched to db myDB
>
```

To confirm the existence of your database, type the command at the MongoDB shell:

```
db
```

```
> db;
myDB
>
```

To get a list of all databases, type the below command:

```
show dbs
```

```
> show dbs;
admin   (empty)
local   0.078GB
test    0.078GB
>
```

Drop database

The syntax to drop database is as follows:

```
db.dropDatabase();
```

To drop the database, “myDB”, first ensure that you are currently placed in “myDB” database and then use the `db.dropDatabase()` command to drop the database.

```
use myDB;
```

```
db.dropDatabase();
```

Confirm if the database “myDB” has been dropped.

```
> db.dropDatabase();  
{ "dropped" : "myDB", "ok" : 1 }  
>
```

If no database is selected, the default database “test” is dropped.

Data types in MongoDB

String	Must be UTF-8 valid. Most commonly used data type. Can be 32-bit or 64-bit (depends on the server).
Integer	To store a true/false value.
Boolean	To store floating point (real values).
Double	To compare a value against the lowest or highest BSON elements.
Min/Max keys	To store arrays or list or multiple values into one key.
Arrays	To record when a document has been modified or added.
Timestamp	To store a NULL value. A NULL is a missing or unknown value.
Null	To store the current date or time in Unix time format. One can create object of date and pass day, month and year to it.
Date	To store the document's id.
Object ID	To store binary data (images, binaries, etc.).
Binary data	To store javascript code into the document.
Code	To store regular expression.
Regular expression	

To report the name of the current database:

```
C:\windows\system32\cmd.exe - mongo
```

```
> db
test
>
```

To display the list of databases:

```
C:\windows\system32\cmd.exe - mongo
```

```
> show dbs
admin (empty)
local 0.078GB
myDB1 0.078GB
>
```

To switch to a new database, for example, myDB1:

```
C:\windows\system32\cmd.exe - mongo
```

```
> use myDB1
switched to db myDB1
>
```

To display the list of collections (tables) in the current database:

```
C:\windows\system32\cmd.exe - mongo
```

```
> show collections
system.indexes
system.js
>
```

To display the current version of the MongoDB server:

```
C:\windows\system32\cmd.exe - mongo
```

```
> db.version()
2.6.1
>
```

To display the statistics that reflect the use state of a database:

```
C:\windows\system32\cmd.exe - mongo
> db.stats()
{
  "db" : "myDB1",
  "collections" : 3,
  "objects" : 6,
  "avgObjSize" : 122.66666666666667,
  "dataSize" : 736,
  "storageSize" : 24576,
  "numExtents" : 3,
  "indexes" : 1,
  "indexSize" : 8176,
  "fileSize" : 67108864,
  "nsSizeMB" : 16,
  "dataFileVersion" : {
    "major" : 4,
    "minor" : 5
  },
  "extentFreeList" : {
    "num" : 14,
    "totalSize" : 974848
  },
  "ok" : 1
}
```

Type in db.help() in the MongoDB client to get a list of commands:

```
C:\windows\system32\cmd.exe - mongo
> db.help()
DB methods:
db.adminCommand(nameOrDocument) - switches to 'admin' db, and runs command [ just calls db.runCommand(...) ]
db.auth(username, password)
db.cloneDatabase(fromhost)
db.commandHelp(name) returns the help for the command
db.copyDatabase(fromdb, todb, fromhost)
db.createCollection(name, { size : ..., capped : ..., max : ... })
db.createUser(userDocument)
db.currentOp() displays currently executing operations in the db
db.dropDatabase()
db.eval(func, args) run code server-side
db.fsyncLock() flush data to disk and lock server for backups
db.fsyncUnlock() unlocks server following a db.fsyncLock()
db.getCollection(cname) same as db['cname'] or db.cname
db.getCollectionNames()
db.getLastErrorMessage() - just returns the err msg string
db.getMongo() - return full status object
db.getMongo() get the server connection object
db.getMongo().setSlaveOk() allow queries on a replication slave server
db.getName()
db.getPrevError()
db.getProfilingLevel() - deprecated
db.getProfilingStatus() - returns if profiling is on and slow threshold
db.getReplicationInfo()
db.getSiblingDB(name) get the db at the same server as this one
db.getWriteConcern() - returns the write concern used for any operations on this db, inherited from server object if set
db.hostInfo() get details about the server's host
db.isMaster() check replica primary status
db.killOp(opid) kills the current operation in the db
db.listCommands() lists all the db commands
db.loadServerScripts() loads all the scripts in db.system.js
db.logout()
db.printCollectionStats()
db.printReplicationInfo()
db.printShardingStatus()
db.printSlaveReplicationInfo()
db.dropUser(username)
db.repairDatabase()
db.resetError()
db.runCommand(cmdobj) run a database command. if cmdobj is a string, turn it into { cmdobj : 1 }
db.serverStatus()
db.setProfilingLevel(level, <slowms>) 0=off 1=slow 2=all
db.setWriteConcern(<write concern doc>) - sets the write concern for writes to the db
db.unsetWriteConcern(<write concern doc>) - unsets the write concern for writes to the db
db.setVerboseShell(flag) display extra information in shell output
db.shutdownServer()
db.stats()
db.version() current version of the server
```

RDBMS	MongoDB
Insert	
Insert into Students (StudRollNo, StudName, Grade, Hobbies, DOJ) Values ('S101', 'Simon David', 'VII', 'Net Surfing', '10-Oct-2012')	db.Students.insert({_id:1, StudRollNo: 'S101', StudName: 'Simon David', Grade: 'VII', Hobbies: 'Net Surfing', DOJ: '10-Oct-2012'});
Update	
Update Students set Hobbies ='Ice Hockey' where StudRollNo ='S101'	db.Students.update({StudRollNo: 'S101'}, {\$set: {Hobbies : 'Ice Hockey'}})
Update Students Set Hobbies ='Ice Hockey'	db.Students.update({},{\$set: {Hobbies: 'Ice Hockey' }}, {multi:true})
Delete	
Delete from Students where StudRollNo = 'S101'	db.Students.remove ({StudRollNo : 'S101'})
Delete From Students	db.Students.remove({})
Select	
Select * from Students	db.Students.find() db.Students.find().pretty()
Select * from students where StudRollNo = 'S101'	db.Students.find({StudRollNo: 'S101'})
Select StudRollNo, StudName, Hobbies from Students	db.Students.find({}, {StudRollNo:1, StudName:1, Hobbies:1, _id:0})
Select StudRollNo, StudName, Hobbies from Students where StudRollNo = 'S101'	db.Students.find({StudRollNo: 'S101'}, {StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})

RDBMS	MongoDB
Select StudRollNo, StudName, Hobbies From Students Where Grade ='VII' and Hobbies ='Ice Hockey'	db.Students.find({Grade: 'VII' , Hobbies: 'Ice Hockey'}, {StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
Select StudRollNo, StudName, Hobbies From Students Where Grade ='VII' or Hobbies = 'Ice Hockey'	db.Students.find({ \$or: [{Grade: 'VII' , Hobbies: 'Ice Hockey'}]}, StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
Select * From Students Where StudName like 'S%'	db.Students.find({ StudName: /^S/}).pretty()

MongoDB query language

RDBMS	MongoDB
Select StudRollNo, StudName, Hobbies From Students Where Grade = 'VII' and Hobbies = 'Ice Hockey'	db.Students.find({Grade: 'VII' , Hobbies: 'Ice Hockey'}, {StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
Select StudRollNo, StudName, Hobbies From Students Where Grade = 'VII' or Hobbies = 'Ice Hockey'	db.Students.find({ \$or: [{Grade: 'VII' , Hobbies: 'Ice Hockey'}]}, StudRollNo : 1, StudName: 1, Hobbies : 1, _id:0})
Select * From Students Where StudName like 'S%'	db.Students.find({ StudName: /^S/}).pretty()

6.5 MONGODB QUERY LANGUAGE

CRUD (Create, Read Update, and Delete) operations in MongoDB

- Create** → Creation of data is done using insert() or update() or save() method.
- Read** → Reading the data is performed using the find() method.
- Update** → Update to data is accomplished using the update() method with UPSERT set to false.
- Delete** → a document is Deleted using the remove() method.

We will present the various methods available in MongoDB shell to deal with data in the next few sections. The sections have been designed as follows:

- Objective:** What is it that we are trying to achieve here?
- Input:** What is the input that has been given to us to act upon?
- Act:** The actual statement/command to accomplish the task at hand.
- Outcome:** The result/output as a consequence of executing the statement.

At few places we have also provided the equivalent in RDBMS such as Oracle.

Objective: To create a collection by the name "Person". Let us take a look at the collection list prior to the creation of the new collection "Person":

```
C:\windows\system32\cmd.exe - mongo
> show collections
students
food
system.indexes
system.js
>
```

Act: The statement to create the collection is
db.createCollection("Person")

```
C:\windows\system32\cmd.exe - mongo
> db.createCollection("Person");
{"ok": 1}
```

Outcome: Below is the collection list after the creation of the new collection "Person":

```
C:\windows\system32\cmd.exe - mongo
> show collections;
Person
Students
food
system.indexes
system.js
```

Objective: To drop a collection by the name "food".
Take a look at the current collection list:

```
C:\windows\system32\cmd.exe - mongo
> show collections;
Person
Students
food
system.indexes
system.js
```

Act: The statement to drop the collection is
db.food.drop();

```
C:\windows\system32\cmd.exe - mongo
> db.food.drop();
true
```

Outcome: The collection list after the execution of the statement is as follows:

```
C:\windows\system32\cmd.exe - mongo
> show collections
Person
Students
system.indexes
system.js
```

4.5.1 Insert Method

We now explain the syntax of insert method.

```
db.students.insert(           ← Collection
{
  RollNo: 101,                ← Field: value
  Age: 19,                    ← Field: value
  ContactNo: 0123456789,      ← Field: value
  EmailID: Sample@abc.com     ← Field: value
})
```

Objective: To create a collection by the name “Students” and insert documents.

Input: Check the list of existing collections.

```
C:\windows\system32\cmd.exe - mongo
> show collections
system.indexes
system.js
>
```

Act: Create a collection by the name “Students” and store the following data in it.

db.Students.insert({_id:1, StudName:"Michelle Jacintha", Grade: "VII", Hobbies: "Internet Surfing"});

```
C:\windows\system32\cmd.exe - mongo
> db.Students.insert({_id:1, StudName:"Michelle Jacintha", Grade: "VII", Hobbies: "Internet Surfing"});
writeResult({ "nInserted" : 1 })
>
```

Outcome: Check if the collection has been successfully created.

```
C:\windows\system32\cmd.exe - mongo
> show collections
Students
system.indexes
system.js
>
```

Check if the document for Student “Michelle Jacintha” has been successfully inserted into the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find();
{"_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
>
```

To format the result, one can add the pretty() method to the operation.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
```

Objective: Insert another document into the collection.

Input: Check the documents in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
```

Act:

```
db.Students.insert({_id:2, StudName:"Mabel Mathews", Grade:"VII", Hobbies:"Baseball"});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.insert({_id:2, StudName:"Mabel Mathews", Grade:"VII", Hobbies:"Baseball"});
WriteResult({ "nInserted" : 1 })
>
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();

  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet surfing"

  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
```

Objective: Insert the document for “Aryan David” into the Students collection only if it does not already exist in the collection. However, if it is already present in the collection, then update the document with new values. (Update his Hobbies from “Skating” to “Chess”.) Use “**Update else insert**” (if there is an existing document, it will attempt to update it, if there is no existing document then it will insert it).

Input: Check the documents in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();

  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet surfing"

  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
```

Act:

```
db.Students.update({_id:3, StudName:"Aryan David", Grade:"VII"},{$set:{Hobbies:"Skating"}},{upsert:true});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:3, StudName:"Aryan David", Grade:"VII"},{$set:{Hobbies:"Skating"}},{upsert:true});
WriteResult({ "nMatched" : 0, "nUpserted" : 1, "nModified" : 0, "_id" : 3 })
```

Outcome: Confirm the presence of the document of “Aryan David” in the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:3});
  "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Skating" }
```

Objective: Insert the document for “Aryan David” into the Students collection only if it does not already exist in the collection. However, if it is already present in the collection, then update the document with new values. (Update his Hobbies from “Skating” to “Chess”). Use “**Update else insert**” (if there is an existing document, it will attempt to update it, if there is no existing document then it will insert it). Try the UPSERT operator by setting it first to “true” and then to “false” and observe the output.

Input: Check the documents in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Skating"
}
```

Act:

```
db.Students.update({_id:3, StudName:"Aryan David", Grade: "VII"},{$set:{Hobbies:
"Chess"}},{upsert:true});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:3, StudName:"Aryan David", Grade: "VII"},{$set:{Hobbies: "Chess"}},{upsert:true});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: Confirm that the required changes have been made to the document of “Aryan David” in the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:3});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
```

Objective: To demonstrate Save method to insert a document for student “Vamsi Bapat” in the “Students” collection. Omit providing value for the _id key.

Input: Check the documents in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
```

Act:

```
db.Students.save({StudName:"Vamsi Bapat",Grade:"VI"})
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.save({StudName:"Vamsi Bapat",Grade:"VI"})
writeResult({ "nInserted" : 1 })
```

Outcome: Confirm the presence of the document of “Vamsi Bapat” in the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().pretty();

  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"

  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"

  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"

  "_id" : ObjectId("546dd0e0a7fba710799bb94d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
```

6.5.2 Save() Method

We now explain the save() method. The save() method will insert a new document if the document with the specified _id does not exist. However, if a document with the specified id exists, it replaces the existing document with the new one.

Objective: Insert the document of “Hersch Gibbs” into the Students collection using the Update method. First try with upsert set to false and then with upsert set to true.

Input: Check the documents in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find();
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : ObjectId("546dd0e0a7fba710799bb94d"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
```

Act: Update method with upsert set to false.

```
db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies:
"Graffiti"}}, {upsert:false});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:false});
writeResult({ "nMatched" : 0, "nUpserted" : 0, "nModified" : 0 })
```

As evident from the above display (nUpserted : 0), no document has been inserted because upsert is set to false.

Update method with upsert set to true.

```
db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies:
"Graffiti"}}, {upsert:true});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade: "VII"},{$set:{Hobbies: "Graffiti"}}, {upsert:true});
WriteResult({ "nMatched" : 0, "nUpserted" : 1, "nModified" : 0, "_id" : 4 })
```

nUpserted: 1 implies that a document with _id:4 has been inserted.

Outcome: Confirm the presence of the document of “Hersch Gibbs” in the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find();
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : ObjectId("546dd0e0a7fba710799bb94d"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
```

6.5.3 Adding a New Field to an Existing Document – Update Method

We now discuss the syntax of update method.

```
db.students.update(  ← Collection
{Age: {$gt 18}},      ← Update Criteria
{$set: {Status: "A"} } ← Update Action
{multi:true}         ← Update Option
)
```

Objective: To add a new field “Location” with value “Newark” to the document (_id:4) of “Students” collection.

Input: Check the document (_id:4) in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

Act:

```
db.Students.update({_id:4},{$set:{Location:"Newark"}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:4},{$set:{Location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: Confirm that the new field “Location” with value “Newark” has been added to document (_id:4) in the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
```

6.5.4 Removing an Existing Field from an Existing Document – Remove Method

In this section we will explain the syntax of remove method.

```
db.students.remove(  ← Collection
{Age: {$gt 18}},      ← Remove Criteria
)
```

Objective: To remove the field “Location” with value “Newark” in the document (_id:4) of “Students” collection.

Input: Check the document (_id:4) in the “Students” collection before proceeding.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({'_id':4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
```

Act:

```
db.Students.update({'_id':4},{ $unset:{Location:"Newark"}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Students.update({'_id':4},{ $unset:{Location:"Newark"}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: Confirm if the stated field (“Location”) has been dropped from the document (_id:4) of the “Students” collection.

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({'_id':4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

6.5.5 Finding Documents based on Search Criteria – Find Method

The syntax of find method is as follows:

```
db.students.find(  ← Collection
{Age: {$gt 18}},    ← Selection Criteria
{RollNo:1, Age:1, _id:1} ← Projection
).limit(10)         ← Cursor Modifier
```

Objective: To search for documents from the “Students” collection based on certain search criteria.

Input: Check the documents in the “Students” collection before proceeding.

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"

  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"

  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"

  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"

  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

Act: Find the document wherein the "StudName" has value "Aryan David".

```
db.Students.find({StudName:"Aryan David"});
```

Outcome:

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:"Aryan David"});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
>

```

To format the above output, use the pretty() method:

```
db.Students.find({StudName:"Aryan David"}).pretty();
```

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:"Aryan David"}).pretty();
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
>

```

RDBMS equivalent:

Select *

From Students

Where StudName like 'Aryan David';

```
SQL> select * from Students where StudName like 'Aryan David';
```

STUDR	STUDNAME	GRADE	HOBBIES
3	Aryan David	VII	Chess

```
SQL>
```

Objective: To display only the StudName from all the documents of the Student's collection. The identifier "__id" should be suppressed and NOT displayed.

Act:

```
db.Students.find({}, {StudName:1, _id:0});
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({}, {StudName:1, _id:0});
{ "StudName" : "Michelle Jacintha" }
{ "StudName" : "Aryan David" }
{ "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat" }
{ "StudName" : "Mabel Mathews" }
```

RDBMS equivalent:

```
Select StudName
From Students;
```

```
SQL> select StudName from Students;
STUDNAME
-----
Michelle Jacintha
Aryan David
Mabel Mathews
Hersch Gibbs
Vamsi Bapat
SQL>
```

Objective: To display only the StudName and Grade from all the documents of the Students collection. The identifier `_id` should be suppressed and NOT displayed.

Act:

```
db.Students.find({}, {StudName:1, Grade:1, _id:0});
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({}, {StudName:1, Grade:1, _id:0});
{ "StudName" : "Michelle Jacintha", "Grade" : "VII" }
{ "Grade" : "VII", "StudName" : "Aryan David" }
{ "Grade" : "VII", "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat", "Grade" : "VI" }
{ "StudName" : "Mabel Mathews", "Grade" : "VII" }
```

RDBMS equivalent:

```
Select StudName, Grade
From Students;
```

```
SQL> select StudName, Grade from Students;
STUDNAME      GRADE
-----
Michelle Jacintha  VII
Aryan David      VII
Mabel Mathews     VII
Hersch Gibbs     VII
Vamsi Bapat      VI
SQL>
```

Objective: To display the StudName, Grade as well the identifier, `_id` from the document of the Students collection where the `_id` column is 1.

Act:

```
db.Students.find({_id:1}, {StudName:1, Grade:1});
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:1},{StudName:1,Grade:1});
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII" }
```

RDBMS equivalent:

```
Select StudRollNo, StudName, Grade
From Students
Where StudRollNo = '1';
```

```
SQL> select StudRollNo, StudName, Grade from Students where StudRollNo = '1';
STUDR STUDNAME      GRADE
-----
1      Michelle Jacintha      VII
SQL>
```

Objective: To display the StudName and Grade from the document of the Students collection where the _id column is 1. The _id field should NOT be displayed.

Act:

```
db.Students.find({_id:1},{StudName:1,Grade:1,_id:0});
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({_id:1},{StudName:1,Grade:1,_id:0});
{ "StudName" : "Michelle Jacintha", "Grade" : "VII" }
```

RDBMS equivalent:

```
Select StudName, Grade
From Students
Where StudRollNo like '1';
```

```
SQL> select StudName, Grade from Students where StudRollNo like '1';
STUDNAME      GRADE
-----
Michelle Jacintha      VII
SQL>
```

Relational operators available to use in the search criteria:

```
$eq    → equal to
$ne    → not equal to
$gte   → greater than or equal to
$lte   → less than or equal to
$gt    → greater than
$lt    → less than
```

Objective: To find those documents where the Grade is set to 'VII'.

Act:

```
db.Students.find({Grade:{ $eq:'VII'}}).pretty();
```

Outcome:

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({Grade:{eq:'VII'}}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 3,
  "StudName" : "Aryan David",
  "Grade" : "VII",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "StudName" : "Hersch Gibbs",
  "Grade" : "VII",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

RDBMS Equivalent:

```

Select *
  From Students
 Where Grade like 'VII';

```

```

SQL> select * from Students where Grade like 'VII';
STUDR STUDNAME          GRADE HOBBIES
-----
1 Michelle Jacintha      VII  Internet surfing
3 Aryan David           VII  Chess
4 Mabel Mathews         VII  Baseball
Hersch Gibbs           VII  Graffiti
SQL>

```

Objective: To find those documents where the Grade is NOT set to 'VII'.

Act:

```
db.Students.find({Grade:{ne:'VII'}}).pretty();
```

Outcome:

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({Grade:{ne:'VII'}}).pretty();
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}

```

There is just one document that meets the above criteria of Grade NOT EQUAL to 'VII'.

RDBMS Equivalent:

```

Select *
  From Students
 Where Grade <> 'VII';

```

```

SQL> select * from Students where Grade <> 'VII';
STUDR STUDNAME          GRADE HOBBIES
-----
Vamsi Bapat           VI
SQL>

```

OR

```
SQL> select * from Students where Grade != 'VII';
```

STUDR	STUDNAME	GRADE	HOBBIES
	Vamsi Bapat	VI	

```
SQL>
```

Objective: To find those documents from the Students collection where the Hobbies is set to either 'Chess' or is set to 'Skating'.

Act:

```
db.Students.find ({Hobbies:{ $in: ['Chess','Skating']}}).pretty ();
```

Outcome:

```
> db.Students.find ({Hobbies:{ $in: ['Chess','Skating']}}).pretty ();
```

```
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
```

RDBMS Equivalent:

```
Select *
  From Students
  Where Hobbies in ('Chess', 'Skating');
```

```
SQL> select * from Students where Hobbies in ('Chess', 'Skating');
```

STUDR	STUDNAME	GRADE	HOBBIES
3	Aryan David	VII	Chess

```
SQL>
```

Objective: To find those documents from the Students collection where the Hobbies is set neither to 'Chess' nor is set to 'Skating'.

Act:

```
db.Students.find ({Hobbies:{ $nin: ['Chess','Skating']}}).pretty ();
```

Outcome:

```
> db.Students.find ({Hobbies:{ $nin: ['Chess','Skating']}}).pretty ();
```

```
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"

  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"

  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"

  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

RDBMS Equivalent:

```
Select *
  From Students
  Where Hobbies not in ('Chess', 'Skating');
```

```
# SQL Plus
SQL> select * from Students where Hobbies not in ('Chess','Skating');
STUDR STUDNAME          GRADE HOBBIES
-----
1      Michelle Jacintha  VII   Internet surfing
2      Mabel Mathews      VII   Baseball
4      Hersch Gibbs       VII   Graffiti
SQL>
```

Objective: To find those documents from the Students collection where the Hobbies is set to 'Graffiti' and the StudName is set to 'Hersch Gibbs' (AND condition).

Act:

```
db.Students.find({Hobbies:'Graffiti', StudName: 'Hersch Gibbs'}).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({Hobbies:'Graffiti', StudName: 'Hersch Gibbs'}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

RDBMS Equivalent:

```
Select *
  From Students
  Where Hobbies like 'Graffiti' and StudName like 'Hersch Gibbs';
```

```
# SQL Plus
SQL> select * from Students where Hobbies like 'Graffiti' and StudName like 'Hersch Gibbs';
STUDR STUDNAME          GRADE HOBBIES
-----
4      Hersch Gibbs       VII   Graffiti
SQL>
```

Objective: To find documents from the Students collection where the StudName begins with "M".

Act:

```
db.Students.find({StudName:/^M/}).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:/^M/}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

RDBMS Equivalent:

```
Select *
  From Students
 Where StudName like 'M%';
```

```
SQL> select * from Students where StudName like 'M%';
STUDR STUDNAME          GRADE HOBBIES
-----
1      Michelle Jacintha VII    Internet surfing
2      Mabel Mathews     VII    Baseball
SQL>
```

Objective: To find documents from the Students collection where the StudName ends in "s".

Act:

```
db.Students.find({StudName:/s$/}).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:/s$/}).pretty();
{
  "id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

RDBMS Equivalent:

```
Select *
  From Students
 Where StudName like '%s';
```

```
SQL> select * from Students where StudName like '%s';
STUDR STUDNAME          GRADE HOBBIES
-----
2      Mabel Mathews     VII    Baseball
4      Hersch Gibbs      VII    Graffiti
SQL>
```

Objective: To find documents from the Students collection where the StudName has an "e" in any position.

Act:

```
db.Students.find({StudName:/e/}).pretty();
```

OR

```
db.Students.find({StudName:/.*e.*$/}).pretty();
```

OR

```
db.Students.find({StudName:{$regex:"e"}}).pretty();
```

Outcome:

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:/e/}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 4,
  "StudName" : "Hersch Gibbs",
  "Grade" : "VII",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

RDBMS Equivalent:

```

Select *
  From Students
  Where StudName like '%e%';

```

```

# SQL Plus
SQL> select * from Students where StudName like '%e%';
STUDR STUDNAME          GRADE HOBBIES
-----
1      Michelle Jacintha  VII   Internet surfing
2      Mabel Mathews     VII   Baseball
4      Hersch Gibbs      VII   Graffiti
SQL>

```

Objective: To find documents from the Students collection where the StudName ends in "a".

Act:

```
db.Students.find({StudName:{$regex:"a$"}}).pretty();
```

Outcome:

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({StudName:{$regex:"a$"}}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}

```

RDBMS Equivalent:

```

Select *
  From Students
  Where StudName like '%a';

```

```

# SQL Plus
SQL> select * from Students where StudName like '%a';
STUDR STUDNAME          GRADE HOBBIES
-----
1      Michelle Jacintha  VII   Internet surfing
SQL>

```

Objective: To find documents from the Students collection where the StudName begins with “M”.

Act:

```
db.Students.find({StudName:{$regex:"^M"}}).pretty();
```

Outcome:

```

C:\mongodb\bin>mongo
> use students
> db.Students.find({StudName:{$regex:"^M"}}).pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

RDBMS Equivalent:

```

Select *
From Students
Where StudName like 'M%';

```

```

SQL> select * from Students where StudName like 'M%';
STUDR STUDNAME          GRADE HOBBIES
-----
1      Michelle Jacintha  VII   Internet surfing
2      Mabel Mathews     VII   Baseball
SQL>

```

6.5.6 Dealing with NULL Values

Objective: To add a new field with null value in existing documents (_id:3 and _id:4) of Students collection. A NULL is a missing or unknown value. When we place NULL as a value for a field, it implies that currently we do not know the value or the value is missing. We can always update the value of the field once we know it.

Input: Before we execute the commands to update documents with a null value in a column, let us first view the two documents.

```
db.Students.find({_id:3},{_id:4}))
```

```

C:\mongodb\bin>mongo
> use students
> db.Students.find({_id:3},{_id:4});
{
  "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess"
}
{
  "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti"
}

```

Act: Update the documents with NULL values in the “Location” column.

```

db.Students.update({_id:3},{set:{Location:null}});
db.Students.update({_id:4},{set:{Location:null}});

```

```

C:\mongodb\bin>mongo
> use students
> db.Students.update({_id:3},{set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Students.update({_id:4},{set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>

```

RDBMS Equivalent:

Update Students

Set Location = null

Where StudRollNo in ('3','4');

```

SQL> update Students set Location = null where StudRollNo in ('3','4');
2 rows updated.
SQL>

```

Outcome: To search for NULL values in Location column.**db.Students.find({Location:{Seq:null}});**

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({Location:{Seq:null}});
{"_id": 1, "StudName": "Michelle Jacintha", "Grade": "VII", "Hobbies": "Internet Surfing" }
{"_id": 3, "Grade": "VII", "StudName": "Aryan David", "Hobbies": "Chess", "Location": null }
{"_id": 4, "Grade": "VII", "StudName": "Hersch Gibbs", "Hobbies": "Graffiti", "Location": null }
{"_id": ObjectId("5464849889ad1ab07d489b7f"), "StudName": "Vamsi Bapat", "Grade": "VI", "Hobbies": "Baseball" }

```

The above statement displays documents which either have NULL values in the location column or do not have the location column at all.

RDBMS Equivalent:

Select *

From Students

Where Location is Null;

```

SQL> select * from Students where Location is NULL;

```

STUDR	STUDNAME	GRADE	HOBBIES	LOCATION
1	Michelle Jacintha	VII	Internet surfing	
3	Aryan David	VII	Chess	
4	Mabel Mathews	VII	Baseball	
	Hersch Gibbs	VII	Graffiti	
	Vamsi Bapat	VI		

```

SQL>

```

Objective: To remove "Location" field having "NULL" values from the documents (_id:3 and _id:4) from the Students collection.

Input: Document from the "Students" collection having "NULL" values in the "Location" column.

```

C:\windows\system32\cmd.exe - mongo
> db.Students.find({Location:{Seq:null}});
{"_id": 1, "StudName": "Michelle Jacintha", "Grade": "VII", "Hobbies": "Internet Surfing" }
{"_id": 3, "Grade": "VII", "StudName": "Aryan David", "Hobbies": "Chess", "Location": null }
{"_id": 4, "Grade": "VII", "StudName": "Hersch Gibbs", "Hobbies": "Graffiti", "Location": null }
{"_id": ObjectId("5464849889ad1ab07d489b7f"), "StudName": "Vamsi Bapat", "Grade": "VI", "Hobbies": "Baseball" }

```

Act:**db.Students.update({_id:3},{Sunset:{Location:null}});****db.Students.update({_id:4},{Sunset:{Location:null}});**

```

C:\windows\system32\cmd.exe - mongo
> db.Students.update({_id:3},{Sunset:{Location:null}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 0 })
> db.Students.update({_id:4},{Sunset:{Location:null}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 0 })

```

Outcome: Let us confirm if the changes have been made by running find method on the Students collection.

```

C:\Users\user>mongo
> use students
> db.Students.find({})
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
{ "_id" : ObjectId("5464849889a91ab07d489b7f"), "StudName" : "Vamsi Bapat", "Grade" : "VII", "Hobbies" : "Reading" }

```

6.5.7 Count, Limit, Sort, and Skip

Objective: To find the number of documents in the Students collection.

Act:

```
db.Students.count()
```

Outcome:

```

C:\Windows\system32\cmd.exe - mongo
> db.Students.count()
4

```

Objective: To find the number of documents in the Students collection wherein the Grade is VII.

Act:

```
db.Students.count({Grade:"VII"});
```

Outcome:

```

C:\Windows\system32\cmd.exe - mongo
> db.Students.count({Grade:"VII"});
4

```

Objective: To retrieve the first 3 documents from the Students collection wherein the Grade is VII.

Act:

```
db.Students.find({Grade:"VII"}).limit(3).pretty();
```

Outcome:

```

C:\Users\user>mongo
> use students
> db.Students.find({Grade:"VII"}).limit(3).pretty();
{ "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing" }
{ "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess" }
{ "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti" }

```

RDBMS Equivalent:

Select *

From Students

Where Grade like 'VII' and rownum < 4;

```
SQL> select * from Students where Grade like 'VII' and rownum < 4;
```

STUDR	STUDNAME	GRADE	HOBBIES	LOCATION
NO	Michelle Jacintha	VII	Internet surfing	
	Aryan David	VII	Chess	
	Mabel Mathews	VII	Baseball	

SQL>

Objective: To sort the documents from the Students collection in the ascending order of StudName.

Act:

```
db.Students.find().sort({StudName:1}).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.Students.find().sort({StudName:1}).pretty();
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

RDBMS Equivalent:

Select *

From Students

Order by StudName asc;

```
SQL> select * from Students order by StudName asc;
```

STUDR	STUDNAME	GRADE	HOBBIES	LOCATION
NO	Aryan David	VII	Chess	
	Hersch Gibbs	VII	Graffiti	
	Mabel Mathews	VII	Baseball	
	Michelle Jacintha	VII	Internet surfing	
	Vamsi Bapat	VI		

SQL>

Objective: To sort the documents from the Students collection in the descending order of StudName.

Act:

```
db.Students.find().sort({StudName:-1}).pretty();
```

Outcome:

```

C:\mongodb\system2\cmd.exe - mongo
> db.Students.find().sort({StudName:-1}).pretty();
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI",
  "Hobbies" : "Internet Surfing",
  "Location" : "Mumbai",
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Baseball",
  "Location" : "Mumbai",
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball",
  "Location" : "Mumbai",
  "_id" : 4,
  "StudName" : "Hersch Gibbs",
  "Grade" : "VII",
  "Hobbies" : "Graffiti",
  "Location" : "Mumbai",
  "_id" : 3,
  "StudName" : "Aryan David",
  "Grade" : "VII",
  "Hobbies" : "Chess",
  "Location" : "Mumbai"
}

```

RDBMS Equivalent:

```

Select *
  From Students
 Order by StudName desc;

```

```

SQL> select * from Students order by StudName desc;
STUDR STUDNAME          GRADE HOBBIES          LOCATION
-----
1      Vamsi Bapat        VI    Internet surfing     Mumbai
2      Michelle Jacintha  VII   Baseball             Mumbai
4      Mabel Mathews        VII   Baseball             Mumbai
3      Hersch Gibbs        VII   Graffiti            Mumbai
3      Aryan David          VII   Chess                Mumbai
SQL>

```

Objective: To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in descending order.

Act:

```
db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
```

Outcome:

```

C:\mongodb\system2\cmd.exe - mongo
> db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI",
  "Hobbies" : "Internet Surfing",
  "Location" : "Mumbai",
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Baseball",
  "Location" : "Mumbai",
  "_id" : 4,
  "StudName" : "Hersch Gibbs",
  "Grade" : "VII",
  "Hobbies" : "Graffiti",
  "Location" : "Mumbai",
  "_id" : 3,
  "StudName" : "Aryan David",
  "Grade" : "VII",
  "Hobbies" : "Chess",
  "Location" : "Mumbai",
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball",
  "Location" : "Mumbai"
}

```

RDBMS Equivalent:

Select *

From Students

Order by Grade asc, hobbies desc;

```
SQL> select * from Students order by Grade asc, Hobbies desc;
```

STUDR	STUDNAME	GRADE	HOBBIES	LOCATION
1	Vamsi Bapat	VI		
4	Michelle Jacintha	VII	Internet surfing	
3	Hersch Gibbs	VII	Graffiti	
2	Aryan David	VII	Chess	
5	Mabel Mathews	VII	Baseball	

```
SQL>
```

Objective: To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in ascending order.

Act:

```
db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
```

Outcome:

```
db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
```

```
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

```
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

```
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
```

```
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

```
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
```

RDBMS Equivalent:

Select *

From Students

Order by Grade asc, Hobbies asc;

```
SQL> select * from Students order by Grade asc, Hobbies asc;
```

STUDR	STUDNAME	GRADE	HOBBIES	LOCATION
2	Vamsi Bapat	VI		
5	Mabel Mathews	VII	Baseball	
3	Aryan David	VII	Chess	
4	Hersch Gibbs	VII	Graffiti	
1	Michelle Jacintha	VII	Internet surfing	

```
SQL>
```

Objective: To skip the first 2 documents from the Students collection.

Act:

```
db.Students.find().skip(2).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe: mongo
> db.Students.find().skip(2).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}

{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}

{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

RDBMS Equivalent:

```
Select StudRollNo, StudName, Grade, Hobbies
  From (Select StudRollNo, StudName, Grade, Hobbies, RowNum as TheRowNum
        From Students)
 Where TheRowNum > 2;
```

```
SQL> Select StudRollNo, StudName, Grade, Hobbies from (Select StudRollNo, StudName, Grade, Hobbies, Rownum as theRownum from Students) where theRownum > 2;
STUDR STUDNAME          GRADE HOBBIES
-----
2      Mabel Mathews      VII   Baseball
4      Hersch Gibbs       VII   Graffiti
Vamsi Bapat             VI
```

Objective: To sort the documents from the Students collection and skip the first document from the output.

Act:

```
db.Students.find().skip(1).pretty().sort({StudName:1});
```

Outcome:

```
C:\windows\system32\cmd.exe: mongo
> db.Students.find().skip(1).pretty().sort({StudName:1});
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}

{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}

{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

RDBMS Equivalent:

```

Select StudRollNo, StudName, Grade, Hobbies
  From (Select StudRollNo, StudName, Grade, Hobbies, RowNum as TheRowNum
        From Students)
 Where TheRowNum > 1
 Order by StudName;

```

STUDR	STUDNAME	GRADE	HOBBIES
3	Aryan David	VII	Chess
4	Hersch Gibbs	VII	Graffiti
2	Mabel Mathews	VII	Baseball
1	Vamsi Bapat	VI	Baseball

Objective: To display the last 2 records from the Students collection.

Act:

```
db.Students.find().pretty().skip(db.Students.count()-2);
```

Outcome:

```

> db.Students.find().pretty().skip(db.Students.count()-2);
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI",
  "Hobbies" : "Baseball"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

Objective: To retrieve the third, fourth, and fifth document from the Students collection.

Act:

```
db.Students.find().pretty().skip(2).limit(3);
```

Outcome:

```

> db.Students.find().pretty().skip(2).limit(3);
{
  "_id" : 4,
  "StudName" : "Hersch Gibbs",
  "Grade" : "VII",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("5464849889ad1ab07d489b7f"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI",
  "Hobbies" : "Baseball"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}

```

6.5.8 Arrays

Objective: To create a collection by the name “food” and then insert documents into the “food” collection. Each document should have a “fruits” array.

Act:

```
db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ] })
db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes' ] });
db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes' ] });
db.food.insert({_id:5,fruits:[ 'orange','grapes' ] });
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ] })
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes' ] });
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes' ] });
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:5,fruits:[ 'orange','grapes' ] });
WriteResult({ "nInserted" : 1 })
```

Outcome: Let us check if these documents are now in the “food” collection.

db.food.find({})

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({})
{"_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{"_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{"_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{"_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{"_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Objective: To find those documents from the “food” collection which has the “fruits array” constituted of “banana”, “apple” and “cherry”.

Act:

db.food.find({fruits:['banana','apple','cherry']}).pretty()

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({fruits:['banana','apple','cherry']}).pretty()
{"_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
```

Objective: To find those documents from the “food” collection which has the “fruits” array having “banana”, as an element.

Act:

db.food.find({fruits:'banana'})

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({'fruits':'banana'})
{"_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{"_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
```

Objective: To find those documents from the “food” collection which have the “fruits” array having “grapes” in the first index position. The index position begins at 0.

Act:

```
db.food.find({'fruits.1':'grapes'})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({'fruits.1':'grapes'})
{"_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Objective: To find those documents from the “food” collection where “grapes” is present in the 2nd index position of the “fruits” array.

Act:

```
db.food.find({'fruits.2':'grapes'})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({'fruits.2':'grapes'})
{"_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{"_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
```

Objective: To find those documents from the “food” collection where the size of the array is two. The size implies that the array holds only 2 values.

Act:

```
db.food.find({'fruits':{'$size:2}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({'fruits':{'$size:2}})
{"_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Objective: To find those documents from the “food” collection where the size of the array is three. The size implies that the array holds only 3 values.

Act:

```
db.food.find({"fruits":{"size:3}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({"fruits":{"size:3}});
{"_id": 1, "fruits": [ "banana", "apple", "cherry" ] }
{"_id": 2, "fruits": [ "orange", "butterfruit", "mango" ] }
{"_id": 3, "fruits": [ "pineapple", "strawberry", "grapes" ] }
{"_id": 4, "fruits": [ "banana", "strawberry", "grapes" ] }
```

Objective: To find the document with (`_id: 1`) from the “food” collection and display the first two elements from the array “fruits”.

Act:

```
db.food.find(_id:1,{"fruits":{"slice:2}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find(_id:1,{"fruits":{"slice:2}})
{"_id": 1, "fruits": [ "banana", "apple" ] }
```

Objective: To find all documents from the “food” collection which have elements “orange” and “grapes” in the array “fruits”.

Act:

```
db.food.find ({fruits: {$all: ["orange", "grapes"]}}).pretty ();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find ({fruits: {$all: ["orange", "grapes"]}}).pretty ();
{"_id": 5, "fruits": [ "orange", "grapes" ] }
```

Objective: To find those documents from the “food” collection which have the element “orange” in the 0th index position in the array “fruits”.

Act:

```
db.food.find({ "fruits.0" : "orange" }).pretty();
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({ "fruits.0" : "orange" }).pretty();
{"_id": 2, "fruits": [ "orange", "butterfruit", "mango" ] }
{"_id": 5, "fruits": [ "orange", "grapes" ] }
```

Objective: To find the document with (`_id: 1`) from the “food” collection and display two elements from the array “fruits”, starting with the element at 0th index position.

Act:

```
db.food.find({_id:1}, {"fruits":{$slice:[0,2]}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:1}, {"fruits":{$slice:[0,2]}})
{ "_id" : 1, "fruits" : [ "banana", "apple" ] }
```

Objective: To find the document with (`_id: 1`) from the “food” collection and display two elements from the array “fruits”, starting with the element at 1st index position.

Act:

```
db.food.find({_id:1}, {"fruits":{$slice:[1,2]}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:1}, {"fruits":{$slice:[1,2]}});
{ "_id" : 1, "fruits" : [ "apple", "cherry" ] }
```

Objective: To find the document with (`_id: 1`) from the “food” collection and display three elements from the array “fruits”, starting with the element at 2nd index position. Since we have only 3 elements in the array “fruits” for the document with `_id:1`, it displays only one element, the element at 2nd index position, that is, “cherry”.

Act:

```
db.food.find({_id:1}, {"fruits":{$slice:[2,3]}})
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:1}, {"fruits":{$slice:[2,3]}});
{ "_id" : 1, "fruits" : [ "cherry" ] }
```

6.5.8.1 Update on the Array

Before we begin the update operations on the “fruits” array of the documents of “food” collection, let us take a look at the documents that we have in the “food” collection:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({}):
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Objective: To update the document with “_id:4” and replace the element present in the 1st index position of the “fruits” array with “apple”.

Act:

```
db.food.update({_id:4},{set: {'fruits.1': 'apple'}})
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:4},{set: {'fruits.1': 'apple'}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: Let us take a look at how this update has changed our document.

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:4});
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }
```

Objective: To update the document with “_id:1” and replace the element “apple” of the “fruits” array with “An apple”.

Act:

```
db.food.update({_id:1, 'fruits':'apple'},{$set: {'fruits.$': 'An apple' }})
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:1, 'fruits':'apple'},{$set: {'fruits.$': 'An apple' }})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: The document after update is as follows.

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:1});
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
```

Objective: To update the document with “_id:2” and push new key value pairs in the “fruits” array.

Act:

```
db.food.update({_id:2},{push: {price: {orange:60,butterfruit:200,mango:120}}})
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:2},{push: {price: {orange:60,butterfruit:200,mango:120}}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
  "_id" : 2,
  "fruits" : [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price" : [
    {
      "orange" : 60,
      "butterfruit" : 200,
      "mango" : 120
    }
  ]
}
```

6.5.8.2 Further Updates to the Array "fruits" ...

Before we do the updates to the documents in the food collection, let us look at the current state:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }

{ "_id" : 2,
  "fruits" : [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price" : {
    "orange" : 60,
    "butterfruit" : 200,
    "mango" : 120
  }
}
```

Objective: To update the document with “_id:4” by adding an element “orange” to the list of elements in the array “fruits”.

Act:

```
db.food.update({_id:4},{addToSet:{fruits:"orange"}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:4},{addToSet:{fruits:"orange"}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: The result after the execution of the statement is as follows.

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes", "orange" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }

{ "_id" : 2,
  "fruits" : [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price" : {
    "orange" : 60,
    "butterfruit" : 200,
    "mango" : 120
  }
}
```

Objective: To update the document with “_id:4” by popping an element from the list of elements present in the array “fruits”. The element popped is the one from the end of the array.

Act:

```
db.food.update({_id:4},{pop:{fruits:1}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:4},{pop:{fruits:1}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: The “food” collection after the execution of the statement is as follows.

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{"_id":1,"fruits":["banana","An apple","cherry"]}
{"_id":3,"fruits":["pineapple","strawberry","grapes"]}
{"_id":4,"fruits":["banana","apple","grapes"]}
{"_id":5,"fruits":["orange","grapes"]}

  "id":2,
  "fruits": [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price": {
    "orange": 60,
    "butterfruit": 200,
    "mango": 120
  }
}
```

Objective: To update the document with “_id:4” by popping an element from the list of elements present in the array “fruits”. The element popped is the one from the beginning of the array.

Act:

```
db.food.update({'_id':4},{$pop:{fruits:-1}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({'_id':4},{$pop:{fruits:-1}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

Outcome: The “food” collection after the execution of the above update statement is as follows.

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{"_id":1,"fruits":["banana","An apple","cherry"]}
{"_id":3,"fruits":["pineapple","strawberry","grapes"]}
{"_id":4,"fruits":["apple","grapes"]}
{"_id":5,"fruits":["orange","grapes"]}

  "id":2,
  "fruits": [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price": {
    "orange": 60,
    "butterfruit": 200,
    "mango": 120
  }
}
```

Objective: To update the document with “_id:3” by popping two elements from the list of elements present in the array “fruits”. The elements popped are “pineapple” and “grapes”.

The document with “_id:3” before the update is

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({'_id':3});
{"_id":3,"fruits":["pineapple","strawberry","grapes"]}
>
```

Act:

```
db.food.update({_id:3},{ $pullAll:{fruits: [ 'pineapple','grapes' ]}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:3},{ $pullAll:{fruits: [ 'pineapple','grapes' ]}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: The document with “_id:3” after the update is as follows:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:4});
{"_id" : 4, "fruits" : [ "apple", "grapes" ] }
```

Objective: To update the documents having “banana” as an element in the array “fruits” and pop out the element “banana” from those documents.

The “food” collection before the update is as follows:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{"_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{"_id" : 3, "fruits" : [ "strawberry" ] }
{"_id" : 4, "fruits" : [ "apple", "grapes" ] }
{"_id" : 5, "fruits" : [ "orange", "grapes" ] }

{"_id" : 2,
  "fruits" : [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price" : {
    "orange" : 60,
    "butterfruit" : 200,
    "mango" : 120
  }
}
```

Act:

```
db.food.update({fruits:'banana'}, {$pull:{fruits:'banana'}})
```

```
C:\windows\system32\cmd.exe - mongo
> db.food.update({fruits:'banana'}, {$pull:{fruits:'banana'}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: The “food” collection after the update is as follows:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find().pretty();
{"_id" : 1, "fruits" : [ "An apple", "cherry" ] }
{"_id" : 3, "fruits" : [ "strawberry" ] }
{"_id" : 4, "fruits" : [ "apple", "grapes" ] }
{"_id" : 5, "fruits" : [ "orange", "grapes" ] }

{"_id" : 2,
  "fruits" : [
    "orange",
    "butterfruit",
    "mango"
  ],
  "price" : {
    "orange" : 60,
    "butterfruit" : 200,
    "mango" : 120
  }
}
```

Objective: To pull out an array element based on index position.

There is no direct way of pulling the array elements by looking up their index numbers. However a workaround is available. The document with “_id:4” in the food collection prior to the update is as follows:

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:4}).pretty();
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
```

Act: The update statement is

```
db.food.update({_id:4}, {$unset: {"fruits.1" : null }}) ;
db.food.update({_id:4}, {$pull : {"fruits" : null}});
```

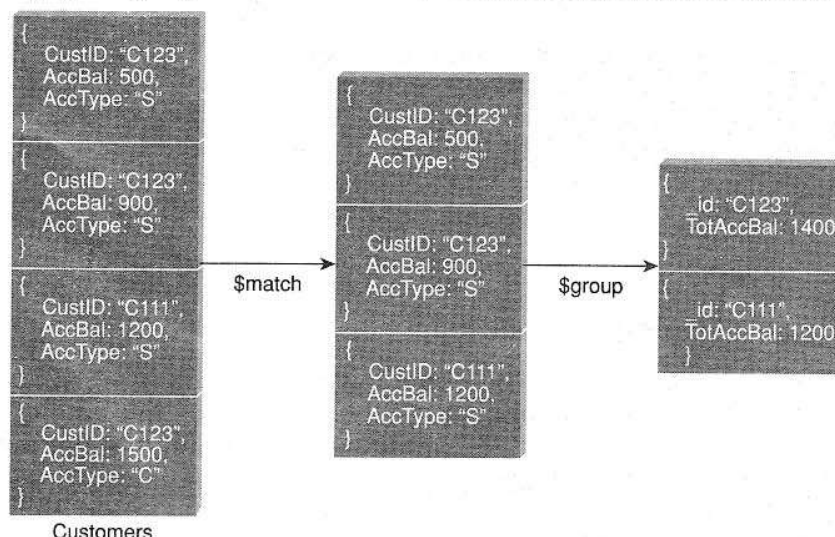
```
C:\windows\system32\cmd.exe - mongo
> db.food.update({_id:4}, {$unset: {"fruits.1" : null }}) ;
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.food.update({_id:4}, {$pull : {"fruits" : null}});
writeResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Outcome: After update, the document with _id:4 in the food collection is

```
C:\windows\system32\cmd.exe - mongo
> db.food.find({_id:4}).pretty();
{ "_id" : 4, "fruits" : [ "apple" ] }
```

6.5.9 Aggregate Function

Objective: Consider the collection “Customers” as given below. It has four documents. We would like to filter out those documents where the “AccType” has a value other than “S”. After the filter, we should be left with three documents where the “AccType”: “S”. It is then required to group the documents on the basis of CustID and sum up the “AccBal” for each unique “CustID”. This is similar to the output received with group by clause in RDBMS. Once the groups have been formed [as per the example below, there will be only two groups: (a) “CustID” : “C123” and (b) “CustID” : “C111”], filter and display that group where the “TotAccBal” column has a value greater than 1200.



Let us start off by creating the collection “Customers” with the above displayed four documents:

```
db.Customers.insert([{"CustID":"C123",AccBal:500,AccType:"S"},
{"CustID":"C123", AccBal: 900, AccType:"S"},
{"CustID":"C111", AccBal: 1200, AccType:"S"},
{"CustID":"C123", AccBal: 1500, AccType:"C"}]);
```

```

> db.Customers.insert([{"CustID":"C123",AccBal:500,AccType:"S"}, {"CustID":"C123",AccBal:900,AccType:"S"},
BulkWriteResult({
  "writeErrors": [],
  "writeConcernErrors": [],
  "inserted": 4,
  "updated": 0,
  "matched": 0,
  "modified": 0,
  "removed": 0,
  "upserted": []
})

```

To confirm the presence of four documents in the “Customers” collection, use the below syntax:

```
db.Customers.find().pretty();
```

```

> db.Customers.find().pretty();
{
  "_id" : ObjectId("54993269f4263d0150bfa72c"),
  "CustID" : "C123",
  "AccBal" : 500,
  "AccType" : "S",

  "_id" : ObjectId("54993269f4263d0150bfa72d"),
  "CustID" : "C123",
  "AccBal" : 900,
  "AccType" : "S",

  "_id" : ObjectId("54993269f4263d0150bfa72e"),
  "CustID" : "C111",
  "AccBal" : 1200,
  "AccType" : "S",

  "_id" : ObjectId("54993269f4263d0150bfa72f"),
  "CustID" : "C123",
  "AccBal" : 1500,
  "AccType" : "C"
}

```

To group on “CustID” and compute the sum of “AccBal”, use the below syntax:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } } );
```

```

> db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } } );
{
  "_id" : "C111", "TotAccBal" : 1200
  "_id" : "C123", "TotAccBal" : 2900
}

```

In order to first filter on “AccType:S” and then group it on “CustID” and then compute the sum of “AccBal”, use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } } );
```

```

> db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } } );
{
  "_id" : "C111", "TotAccBal" : 1200
  "_id" : "C123", "TotAccBal" : 1400
}

```

In order to first filter on “AccType:S” and then group it on “CustID” and then to compute the sum of “AccBal” and then filter those documents wherein the “TotAccBal” is greater than 1200, use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
{ $match : {TotAccBal : { $gt : 1200 } } } );
```

```

> db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
{ $match : {TotAccBal : { $gt : 1200 } } } );
{
  "_id" : "C123", "TotAccBal" : 1400
}

```

To group on “CustID” and compute the average of the “AccBal” for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID", TotAccBal : { $avg : "$AccBal" } } } );
```

```
{
  "_id" : "C111", "TotAccBal" : 1200
}
{
  "_id" : "C123", "TotAccBal" : 966.6666666666666
}
```

To group on “CustID” and determine the maximum “AccBal” for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID", TotAccBal : { $max : "$AccBal" } } } );
```

```
{
  "_id" : "C111", "TotAccBal" : 1200
}
{
  "_id" : "C123", "TotAccBal" : 1500
}
```

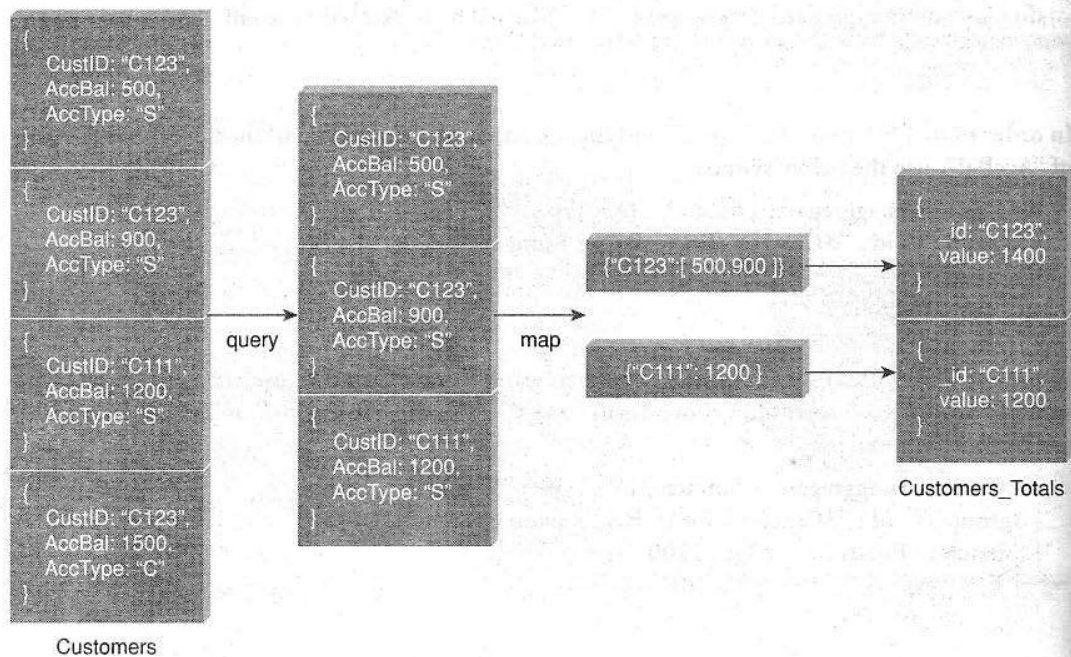
To group on “CustID” and determine the minimum “AccBal” for each group:

```
db.Customers.aggregate( { $group : { _id : "$CustID", TotAccBal : { $min : "$AccBal" } } } );
```

```
{
  "_id" : "C111", "TotAccBal" : 1200
}
{
  "_id" : "C123", "TotAccBal" : 500
}
```

6.5.10 MapReduce Function

Objective: Consider the collection “Customers” below. There are four documents. Run a query to filter out those documents where the key “AccType” has a value other than “S”. Then for each unique CustID, prepare a list of AccBal values. For example, for CustID: “C123”, the AccBals are 500,900. This task will be assigned to the mapper function. The output from the mapper function serves as the input to the reducer function. The reducer function then aggregates the AccBal for each CustID. For example, for CustID: “C123”, the value is 1400, etc.



Given below is the syntax that we will use to accomplish the objective.

```
db.Customers.mapReduce (
  map      →    function() { emit ( this.CustID, this.AccBal ); },
  reduce   →    function(key, values) { return Array.sum (values ) },
  query    →    { query: { AccType: "S"},
  output   →    out: "Customer_Totals"
  )
}
```

Map Function

```
var map = function(){
  emit ( this.CustID, this.AccBal );}
```

```
C:\windows\system32\cmd.exe - mongo
> var map = function(){
... emit ( this.CustID, this.AccBal );}
```

Reduce Function

```
var reduce = function(key, values){ return Array.sum(values) ; }
```

```
C:\windows\system32\cmd.exe - mongo
> var reduce = function(key, values){ return Array.sum(values) ; }
```

To execute the query

```
db.Customers.mapReduce(map, reduce,{out: "Customer_Totals", query:{AccType:"S"}});
```

```
C:\windows\system32\cmd.exe - mongo
> db.Customers.mapReduce(map, reduce,{out: "Customer_Totals", query:{AccType:"S"}});
{
  "result" : "Customer_Totals",
  "timeMillis" : 7,
  "counts" : {
    "input" : 3,
    "emit" : 3,
    "reduce" : 1,
    "output" : 2
  },
  "ok" : 1,
}
```

The output as archived in "Customer_Totals" collection:

```
C:\windows\system32\cmd.exe - mongo
> db.Customer_Totals.find().pretty();
{ "_id" : "C111", "value" : 1200 }
{ "_id" : "C123", "value" : 1400 }
```

6.5.11 Java Script Programming

Objective: To compute the factorial of a given positive number. The user is required to create a function by the name "factorial" and insert it into the "system.js" collection.

Before we proceed, a quick check on what is contained in the "system.js" collection:

```
C:\windows\system32\cmd.exe - mongo
> db.system.js.find();
```

As per the screenshot above, currently there are no functions in the system.js collection.

Act:

```
db.system.js.insert({_id:"factorial",
```

```

value:function(n)
{
    if (n==1)
    return 1;
    else
    return n * factorial(n-1);
}
}
);

```

```

C:\windows\system32\cmd.exe - mongo
> db.system.js.insert({_id:"factorial",
... value:function(n)
... {
...   if (n==1)
...   return 1;
...   else
...   return n * factorial(n-1);
... }
... });
WriteResult({"nInserted" : 1 })

```

Confirm the presence of the “factorial” function in the system.js collection.

```

C:\windows\system32\cmd.exe - mongo
> db.system.js.find();
{ "_id" : "factorial", "value" : function (n)
{
  if (n==1)
  return 1;
  else
  return n * factorial(n-1);
} }

```

To execute the function “factorial”, use the eval() method.

```
db.eval("factorial(3)");
```

```

C:\windows\system32\cmd.exe - mongo
> db.eval("factorial(3)");
3

```

```
db.eval("factorial(5)");
```

```

C:\windows\system32\cmd.exe - mongo
> db.eval("factorial(5)");
120

```

```
db.eval("factorial(1)");
```

```

C:\windows\system32\cmd.exe - mongo
> db.eval("factorial(1)");
1

```

6.5.12 Cursors in MongoDB

Objective: To create a collection by the name “alphabets” and insert documents in it containing two fields, “_id” and “alphabet”. The values stored in the “alphabet” field should be “a”, “b”, “c”, “d”, etc. with one value stored per document. There should be 26 documents in all. We need to use cursor to iterate through the “alphabets” collection.

Note: “Alphabets” is the name of the collection and “alphabet” is the name of the field.

Act: To create the collection “alphabets” with its 26 documents.

```

db.alphabets.insert({_id:1,alphabet:"a"});
db.alphabets.insert({_id:2,alphabet:"b"});

```

```
db.alphabets.insert({_id:3,alphabet:"c"});
db.alphabets.insert({_id:4,alphabet:"d"});
db.alphabets.insert({_id:5,alphabet:"e"});
db.alphabets.insert({_id:6,alphabet:"f"});
db.alphabets.insert({_id:7,alphabet:"g"});
db.alphabets.insert({_id:8,alphabet:"h"});
db.alphabets.insert({_id:9,alphabet:"i"});
db.alphabets.insert({_id:10,alphabet:"j"});
db.alphabets.insert({_id:11,alphabet:"k"});
db.alphabets.insert({_id:12,alphabet:"l"});
db.alphabets.insert({_id:13,alphabet:"m"});
db.alphabets.insert({_id:14,alphabet:"n"});
db.alphabets.insert({_id:15,alphabet:"o"});
db.alphabets.insert({_id:16,alphabet:"p"});
db.alphabets.insert({_id:17,alphabet:"q"});
db.alphabets.insert({_id:18,alphabet:"r"});
db.alphabets.insert({_id:19,alphabet:"s"});
db.alphabets.insert({_id:20,alphabet:"t"});
db.alphabets.insert({_id:21,alphabet:"u"});
db.alphabets.insert({_id:22,alphabet:"v"});
db.alphabets.insert({_id:23,alphabet:"w"});
db.alphabets.insert({_id:24,alphabet:"x"});
db.alphabets.insert({_id:25,alphabet:"y"});
db.alphabets.insert({_id:26,alphabet:"z"});
```

```
C:\windows\system32\cmd.exe - mongo
> db.alphabets.insert({_id:1,alphabet:"a"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:2,alphabet:"b"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:3,alphabet:"c"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:4,alphabet:"d"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:5,alphabet:"e"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:6,alphabet:"f"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:7,alphabet:"g"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:8,alphabet:"h"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:9,alphabet:"i"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:10,alphabet:"j"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:11,alphabet:"k"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:12,alphabet:"l"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:13,alphabet:"m"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:14,alphabet:"n"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:15,alphabet:"o"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:16,alphabet:"p"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:17,alphabet:"q"});
WriteResult({"nInserted" : 1 })
> db.alphabets.insert({_id:18,alphabet:"r"});
WriteResult({"nInserted" : 1 })
```

```

> db.alphabets.insert({_id:19,alphabet:"s"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:21,alphabet:"u"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:22,alphabet:"v"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:23,alphabet:"w"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:24,alphabet:"x"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:25,alphabet:"y"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:26,alphabet:"z"});
WriteResult({ "nInserted" : 1 })

```

Confirm the presence of 26 documents in the “alphabets” collection.

```

C:\windows\system32\cmd.exe - mongo
> db.alphabets.find();
{"_id":1,"alphabet":"a"}
{"_id":2,"alphabet":"b"}
{"_id":3,"alphabet":"c"}
{"_id":4,"alphabet":"d"}
{"_id":5,"alphabet":"e"}
{"_id":6,"alphabet":"f"}
{"_id":7,"alphabet":"g"}
{"_id":8,"alphabet":"h"}
{"_id":9,"alphabet":"i"}
{"_id":10,"alphabet":"j"}
{"_id":11,"alphabet":"k"}
{"_id":12,"alphabet":"l"}
{"_id":13,"alphabet":"m"}
{"_id":14,"alphabet":"n"}
{"_id":15,"alphabet":"o"}
{"_id":16,"alphabet":"p"}
{"_id":17,"alphabet":"q"}
{"_id":18,"alphabet":"r"}
{"_id":19,"alphabet":"s"}
{"_id":20,"alphabet":"t"}
Type "it" for more

```

A quick word on how the `db.collection.find()` method works. This is the primary method for read operation. In other words, it allows one to fetch the documents from the collection. To be able to access the documents, one needs to iterate the cursor.

However, in the mongo shell, if the returned cursor is not assigned to a variable using the `var` keyword, then cursor is automatically iterated up to 20 times to print the first 20 documents in the result.

```

C:\windows\system32\cmd.exe - mongo
> var myCursor = db.alphabets.find();
myCursor:
{"_id":1,"alphabet":"a"}
{"_id":2,"alphabet":"b"}
{"_id":3,"alphabet":"c"}
{"_id":4,"alphabet":"d"}
{"_id":5,"alphabet":"e"}
{"_id":6,"alphabet":"f"}
{"_id":7,"alphabet":"g"}
{"_id":8,"alphabet":"h"}
{"_id":9,"alphabet":"i"}
{"_id":10,"alphabet":"j"}
{"_id":11,"alphabet":"k"}
{"_id":12,"alphabet":"l"}
{"_id":13,"alphabet":"m"}
{"_id":14,"alphabet":"n"}
{"_id":15,"alphabet":"o"}
{"_id":16,"alphabet":"p"}
{"_id":17,"alphabet":"q"}
{"_id":18,"alphabet":"r"}
{"_id":19,"alphabet":"s"}
{"_id":20,"alphabet":"t"}
Type "it" for more

```

Let us now look at designing manual cursors to iterate through the documents in the “alphabets” collection. We will use two methods with manual cursors: `hasNext()` and `next()`. We now quickly explain the two methods.

Method 1: `hasNext()` method. Return value: Boolean. The `hasNext()` method returns true if the cursor returned by the `db.Collection.find()` query can iterate further to return more documents.

Method 2: `next()` method. The `next()` method returns the next document in the cursor as returned by the `db.collection.find()` method.

```
C:\windows\system32\cmd.exe - mongo
> var myCur=db.alphabets.find({});
> while(myCur.hasNext()){
...   var myRec=myCur.next();
...   print("The alphabet is : " + myRec.alphabet);
... }
The alphabet is : a
The alphabet is : b
The alphabet is : c
The alphabet is : d
The alphabet is : e
The alphabet is : f
The alphabet is : g
The alphabet is : h
The alphabet is : i
The alphabet is : j
The alphabet is : k
The alphabet is : l
The alphabet is : m
The alphabet is : n
The alphabet is : o
The alphabet is : p
The alphabet is : q
The alphabet is : r
The alphabet is : s
The alphabet is : t
The alphabet is : u
The alphabet is : v
The alphabet is : w
The alphabet is : x
The alphabet is : y
The alphabet is : z
>
```

The same result can be obtained by iterating through the cursor using a `forEach` loop.

```
C:\windows\system32\cmd.exe - mongo
> var cur=db.alphabets.find({});
> var myRec;
> cur.forEach( function(myRec) {
...   print("The alphabet is : " + myRec.alphabet);
... });
The alphabet is : a
The alphabet is : b
The alphabet is : c
The alphabet is : d
The alphabet is : e
The alphabet is : f
The alphabet is : g
The alphabet is : h
The alphabet is : i
The alphabet is : j
The alphabet is : k
The alphabet is : l
The alphabet is : m
The alphabet is : n
The alphabet is : o
The alphabet is : p
The alphabet is : q
The alphabet is : r
The alphabet is : s
The alphabet is : t
The alphabet is : u
The alphabet is : v
The alphabet is : w
The alphabet is : x
The alphabet is : y
The alphabet is : z
>
```

Indexes in MongoDB

What is an Index in MongoDB?

- An **index** in MongoDB is a special data structure that stores a small portion of the data set in an easy-to-traverse form.
- It helps **improve the speed of query operations** by allowing MongoDB to find and retrieve specific documents faster, similar to the index in a book.

Syntax to Create an Index

```
db.collection.createIndex({ fieldName: 1 }) // 1 for ascending, -1 for descending
```

Consider a `students` collection:

```
{
  "_id": 1,
  "name": "Alice",
  "age": 22,
  "department": "AI"
}
```

Create an index on the `name` field:

```
db.students.createIndex({ name: 1 })
```

This will help MongoDB quickly find students by name:

```
db.students.find({ name: "Alice" })
```

Index Type	Description
Single Field	Indexes on one field (e.g., { name: 1 })
Compound	Indexes on multiple fields (e.g., { name: 1, age: -1 })
Multikey	Indexes on array fields
Text	Full-text search on string content
Hashed	For sharded collections using hash-based partitioning
Wildcard	Indexes on dynamic or unknown field names

Method	Description
<code>createIndex()</code>	Creates an index
<code>getIndexes()</code>	Lists all indexes in a collection
<code>dropIndex()</code>	Drops a specific index
<code>dropIndexes()</code>	Drops all indexes except <code>_id</code>
<code>reIndex()</code>	Rebuilds all indexes (deprecated in many cases)
<code>explain()</code>	Shows how MongoDB uses indexes for a query

Assume the collection with the following documents:

```

> db.books.find().pretty();
{
  "_id" : 6,
  "Category" : "Machine Learning",
  "Bookname" : "Machine Learning for Hackers",
  "Author" : "Drew Conway",
  "qty" : 25,
  "price" : 400,
  "rol" : 30,
  "pages" : 350
}

{
  "_id" : 7,
  "Category" : "Web Mining",
  "Bookname" : "Mining the Social Web",
  "Author" : "Matthew A.Russell",
  "qty" : 55,
  "price" : 500,
  "rol" : 30,
  "pages" : 250
}

{
  "_id" : 8,
  "Category" : "Python",
  "Bookname" : "Python for Data Analysis",
  "Author" : "Wes McKinney",
  "qty" : 8,
  "price" : 150,
  "rol" : 20,
  "pages" : 150
}

{
  "_id" : 9,
  "Category" : "Visualization",
  "Bookname" : "Visualizing Data",
  "Author" : "Ben Fry",
  "qty" : 12,
  "price" : 325,
  "rol" : 6,
  "pages" : 450
}

{
  "_id" : 10,
  "Category" : "Web Mining",
  "Bookname" : "Algorithms for the intelligent web",
  "Author" : "Haralambos Marmanis",
  "qty" : 5,
  "price" : 850,
  "rol" : 10,
  "pages" : 120
}

```

Create an index on the key "Category" in the "books" collection.

```
> db.books.ensureIndex({"Category":1});
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 1,
  "numIndexesAfter" : 2,
  "ok" : 1
}
```

Check on the status, that is, number and name of the indexes:

```
> db.books.stats();
{
  "ns" : "test.books",
  "count" : 5,
  "size" : 1200,
  "avgObjSize" : 240,
  "storageSize" : 8192,
  "numExtents" : 1,
  "nindexes" : 2,
  "lastExtentSize" : 8192,
  "paddingFactor" : 1,
  "systemFlags" : 1,
  "userFlags" : 1,
  "totalIndexSize" : 16352,
  "indexSizes" : {
    "_id_" : 8176,
    "Category_1" : 8176
  },
  "ok" : 1
}
```

Get the list of all indexes on the "books" collection:

```
> db.books.getIndexes();
{
  "v" : 1,
  "key" : { "_id" : 1 },
  "name" : "_id_",
  "ns" : "test.books"
},
{
  "v" : 1,
  "key" : { "category" : 1 },
  "name" : "Category_1",
  "ns" : "test.books"
}
```

Instructs mongoDB to use the index on the "category" field in ascending order (1 means ascending order)

To use the index on "Category" in the "books" collection, use the hint method:

```
> db.books.find({"Category": "Web Mining"}).pretty().hint({"Category":1});
{
  "_id" : 7,
  "Category" : "Web Mining",
  "Bookname" : "Mining the Social Web",
  "Author" : "Matthew A.Russell",
  "qty" : 55,
  "price" : 500,
  "rol" : 30,
  "pages" : 250
},
{
  "_id" : 10,
  "Category" : "Web Mining",
  "Bookname" : "Algorithms for the intelligent web",
  "Author" : "Haralambos Marmanis",
  "qty" : 5,
  "price" : 850,
  "rol" : 10,
  "pages" : 120
}
```

Check the explain plan to get a deeper understanding on the use of index.

```
> db.books.find({"Category":"Web Mining"}).pretty().hint({"Category":1}).explain();
{
  "cursor" : "BtreeCursor Category_1",
  "isMultiKey" : false,
  "n" : 2,
  "nscannedObjects" : 2,
  "nscanned" : 2,
  "nscannedObjectsAllPlans" : 2,
  "nscannedAllPlans" : 2,
  "scanAndOrder" : false,
  "indexOnly" : false,
  "nYields" : 0,
  "nChunkSkips" : 0,
  "millis" : 0,
  "indexBounds" : {
    "Category" : [
      [
        "Web Mining",
        "Web Mining"
      ]
    ]
  },
  "server" : "PUNITP123103L:27017",
  "filterSet" : false
}
```

Field	Description
"cursor" : "BtreeCursor Category_1"	MongoDB is using a B-tree index on the field <code>Category</code> . <code>Category_1</code> is the name of the index (automatically named by MongoDB as <code>fieldname_direction</code>).
"isMultiKey" : false	Indicates whether the index used is a multikey index (used for array fields). Here, it's <code>false</code> , meaning the index is on a single value.
"n" : 2	The number of documents returned by the query.
"nscannedObjects" : 0	Number of documents scanned directly. Since this is a covered index query, documents were not scanned.
"nscanned" : 2	Number of index entries scanned . Here, 2 index entries were examined.
"nscannedObjectsAllPlans" : 0	Number of documents scanned in all considered query plans. Here, it's 0 because only index was used.
"nscannedAllPlans" : 2	Number of index entries scanned in all plans.
"scanAndOrder" : false	Indicates if MongoDB needed to scan all documents and then sort them. <code>false</code> means sorting wasn't needed as index was already sorted.

"indexOnly" : true	 Very important! This indicates a Covered Index Query , meaning MongoDB answered the query using only the index without accessing the actual documents .
"nYields" : 0	Number of times the query yielded to allow other operations (used in long-running queries). Here, it's 0.
"nChunkSkips" : 0	Relevant in sharded collections, indicates how many chunks were skipped. Not applicable here.
"millis" : 0	Time taken to execute the query (in milliseconds). This query was very fast.
"indexBounds" : { "Category" : [["Web Mining", "Web Mining"]] }	Shows the range of index keys scanned. Here, it is exactly "Web Mining" to "Web Mining", meaning an exact match.
"server" : "PUNITP123103L:27017"	The server and port where the query was executed.
"filterSet" : false	Internal flag related to the query planner. Not commonly used in practice.

Let us look at the case of covered index. Observe that the "indexOnly" property will be set to true for covered index.

```
> db.books.find({"Category": "Web Mining"}, {"Category": 1, _id: 0}).pretty().hint({"Category": 1}).explain();
{
  "cursor" : "BtreeCursor Category_1",
  "isMultiKey" : false,
  "n" : 2,
  "nscannedObjects" : 0,
  "nscanned" : 2,
  "nscannedObjectsAllPlans" : 0,
  "nscannedAllPlans" : 2,
  "scanAndOrder" : false,
  "indexOnly" : true,
  "nYields" : 0,
  "nChunkSkips" : 0,
  "millis" : 0,
  "indexBounds" : {
    "Category" : [
      [
        "Web Mining",
        "Web Mining"
      ]
    ]
  },
  "server" : "PUNITP123103L:27017",
  "filterSet" : false
}
```

"Category": 1 → **Include** the Category field in the output.

"_id": 0 → **Exclude** the default _id field (which MongoDB includes by default).

If your document looks like this:

```
{
  "_id": 101,
  "Title": "Intro to AI",
  "Category": "Web Mining",
  "Author": "John Doe"
}
```

With { "Category": 1, _id: 0 }, your output will be:

```
{
  "Category": "Web Mining"
}
```

MongoDB Import

6.5.14 Mongolimport

This command used at the command prompt imports CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents into MongoDB.

Objective: Given a CSV file “sample.txt” in the D: drive, import the file into the MongoDB collection, “SampleJSON”. The collection is in the database “test”.

The “sample.txt” file is as follows:

```
_id,FName,LName  
1,Samuel,Jones  
2,Virat,Kumar  
3,Raul,"A Simpson"  
4,,"Andrew Simon"
```

Mongoimport --db test --collection SampleJSON --type csv --headerline --file d:\sample.txt

mongoimport

Tool used to import data into MongoDB.

--db test

Specifies the database to import into. Here, it's the `test` database.

--collection SampleJSON

Specifies the target **collection name** — `SampleJSON` in this case.

--type csv

Indicates the **file type** is CSV (Comma-Separated Values).

--headerline

Uses the **first line of the CSV file** as field names (instead of *field1*, *field2*, etc.).

--file d:\sample.txt

Path to the **file to be imported**. Here it's located at `d:\sample.txt`.

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
2015-02-20T21:09:27.301+0530 imported 4 objects
```

Output: To confirm the output, log into MongoDB shell and navigate to the “SampleJSON” collection in the “test” database.

The following are the JSON documents in the collection:

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.SampleJSON.find().pretty();
{ "_id" : 1, "FName" : "Samuel", "LName" : "Jones" }
{ "_id" : 2, "FName" : "Virat", "LName" : "Kumar" }
{ "_id" : 3, "FName" : "Raul", "LName" : "A Simpson" }
{ "_id" : 4, "FName" : "", "LName" : "Andrew Simon" }
```

6.5.15 MongoExport

This command used at the command prompt exports MongoDB JSON documents into CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents.

Objective: This command used at the command prompt exports MongoDB JSON documents from “Customers” collection in the “test” database into a CSV file “Output.txt” in the D: drive.

Given below is a snapshot of the JSON documents in the “Customers” collection of the “test” database.

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.Customers.find().pretty();
```

```
{
  "_id" : ObjectId("54df6d4f46a31d28183b9a5b"),
  "CustID" : "C123",
  "AccBal" : 500,
  "AccType" : "S"
}
{
  "_id" : ObjectId("54df6d4f46a31d28183b9a5c"),
  "CustID" : "C123",
  "AccBal" : 900,
  "AccType" : "S"
}
```

```
{
  "_id" : ObjectId("54df6d4f46a31d28183b9a5d"),
  "CustID" : "C111",
  "AccBal" : 1200,
  "AccType" : "S"
}
{
  "_id" : ObjectId("54df6d4f46a31d28183b9a5e"),
  "CustID" : "C123",
  "AccBal" : 1500,
  "AccType" : "C"
}
```

```
Mongoexport --db test --collection Customers --csv --fieldFile d:\fields.txt --out d:\output.txt
```

Option	Explanation
<code>mongoexport</code>	MongoDB tool used to export data from a collection.
<code>--db test</code>	The database name — here, it's <code>test</code> .
<code>--collection Customers</code>	The collection name from which data is exported — here, <code>Customers</code> .
<code>--csv</code>	Export the data in CSV format .
<code>--fieldFile d:\fields.txt</code>	Specifies the fields/columns to export , listed in the file <code>fields.txt</code> .
<code>--out d:\output.txt</code>	Specifies the output file path where data will be saved (<code>output.txt</code>).

Before executing this command, ensure that you have created a “fields.txt” with a format defined as follows. The “fields.txt” file:

```
CustID
AccBal
AccType
```

For the MongoExport command to execute successfully, ensure that the fields are spelt as is in the MongoDB collection. The case also has to be maintained. It is mandatory to ensure that only one field name is placed per line.

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
exported 4 records
```

Output: To confirm the output, navigate to the D: drive and check the file “Output.txt”.

```
“Output.txt”
CustID,AccBal,AccType
“C123”,500.0,“S”
“C123”,900.0,“S”
“C111”,1200.0,“S”
“C123”,1500.0,“C”
```

6.5.16 Automatic Generation of Unique Numbers for the “_id” Field

Step 1: Initialize the Counter Document

```
db.usercounters.insert(  
{  
  _id: "empid",  
  seq: 0  
})
```

- A new collection `usercounters` is created.
- A document with `_id: "empid"` and `seq: 0` is inserted.
- This document will **track the last used value** for employee IDs.
- Initially, `seq` is 0. This will be incremented every time a new employee is inserted.

Step 2: Create the Function `getNextseq(name)`

```
function getNextseq(name) {  
  var ret = db.usercounters.findAndModify({  
    query: { _id: name },  
    update: { $inc: { seq: 1 } },  
    new: true  
  });  
  return ret.seq;  
}
```

This function:

- Takes a `name` (like `"empid"`) to identify the right counter document.
- Uses `findAndModify()` to:
 - Find the document with `_id: "empid"`,
 - Increment `seq` by 1 (`$inc: { seq: 1 }`),
 - Return the **updated** document (`new: true`).

Returns the **new seq value** which will be used as the new ID.

Step 3: Insert into Actual Collection Using Auto-Increment ID

```
db.users.insert(  
{  
  _id: getNextseq("empid"),  
  Name: "sarah jane"  
}  
)
```

- A new document is inserted into the `users` collection.
- The `_id` is set using the value returned by `getNextseq("empid")`.
- This ensures that each user inserted gets a **unique, incrementing ID**.

Who is responsible for generating the `_id`?

💡 MongoDB is responsible for automatically generating the `_id` value when it is not provided by the user.

By default, MongoDB assigns a unique **ObjectId** as the `_id`.

This ensures that every document in a collection has a unique identifier.

save() method

1. If the document does NOT have an `_id` field:

- MongoDB inserts it as a new document.
- MongoDB automatically generates a unique `_id`.

2. If the document HAS an `_id` field:

- MongoDB checks if a document with that `_id` already exists.
- If it exists, MongoDB replaces the entire document with the new one.
- If it does not exist, MongoDB inserts it as a new document.

```
db.Students.save({  
  name: "Vamsi Bapat",  
  course: "B.Tech"  
})
```

```
db.Students.save({  
  _id: ObjectId("662062a5a1d3b8ab129c2461"),  
  name: "Vamsi Bapat",  
  course: "M.Tech"  
})
```

In modern MongoDB versions, `save()` is deprecated. It's recommended to use:

- `insertOne()` for inserting,
- `updateOne()` or `replaceOne()` for updating.

Keyword `$all` :

`$all` is an operator used **to match array fields that contain `_all_`** the specified elements****, regardless of the order or presence of other elements.

`'fruits.$'` uses the **positional operator** `$`, which refers to the **first array element that matches** the query condition (`'fruits': 'apple'`).

So, it updates the matching `"apple"` in the array to `"An apple"`.

`$match` : Filtering Stage

Purpose: `$match` filters documents **before processing** them further.

`$group` : Grouping and Aggregation Stage

Purpose: `$group` is used to **group documents by a field** and perform aggregation operations like

`$sum` , `$avg` , `$max` , etc.